Lab 1 – Assembly Programming

IS1200/IS1500 staff

2026

Contents

1 Introduction **2**
- 1.1 Lab Environment . . . 2
- 1.2 Preparations . . . 2
- 1.3 Resources . . . 2
- 1.4 Reading Guidelines . . . 3

2 Examination **3**

3 RARS Setup **4**

4 Assignments **5**
- 4.1 Assignment 1: Analyzing Assembly Code . . . 5
- 4.2 Assignment 2: Writing Your First Assembly Language Function . . . 6
- 4.3 Assignment 3: Printing the Time . . . 7
- 4.4 Assignment 4: Programming a Simple Delay . . . 9
- 4.5 Assignment 5: Move to the Development Board . . . 10
- 4.6 Assignment 6: At the Lab Session . . . 11
- 4.7 Assignment 7: Surprise Assignment . . . 13

5 ASCII Table – Hexadecimal Codes and Corresponding ASCII Characters **14**

6 Version History **15**

1 Introduction

Welcome to the first lab in the course! In this laboratory exercise you will learn the fundamentals of programming in an assembly language. After finishing this lab, you should be able to

1. Analyze RISC-V assembly code by using a simulator.
2. Write short assembly code functions in RISC-V assembler.
3. Execute and test a compiled program on a DTEK-V board.

1.1 Lab Environment

The first part of this lab uses RARS (RISC-V Assembler and Runtime Simulator). Link to the project's repo: https://github.com/TheThirdOne/rars.

In the second part of the lab, you will use a DTEK-V development board, which includes a RISC-V/32 processor. These development boards will be loaned out during one of the early lectures.

To access the lab environment, either using your own computer or through KTH computers, please visit the "Software Resources" page on Canvas.

1.2 Preparations

You must do the lab work in advance, except for a few specific tasks. The teachers will examine your skills and knowledge at the lab session.

> Before your booked lab session, upload your solution files to the Canvas assignment **Lab 1 Files + Upload code** – the assignment page lists exactly which files. When you log in on the KTH lab computers at the session, your submitted code is downloaded automatically, and you explain it to the teachers there. If you modify your code during the session, you may re-upload it to Canvas; re-uploads end up in the "Files" section of your group page.

Booking opens one week before each lab session. We recommend that you book as soon as booking opens, and start preparing at least a week before the session.

Assignments 1 through 5 must be prepared completely before the start of your lab session.

Assignments 6 and 7 are performed during your lab session.

You can ask for help any time before the lab session. There are several ways to get help; see the course website for details and alternatives.

During the lab session, the teachers will perform examinations, but can also offer help. Make sure to state clearly that you want to *ask questions*, rather than being examined.

Sometimes, our teachers may have to refer help-seekers to other sources of information, so that other students can be examined.

1.3 Resources

You will use the RARS simulator for Assignments 1 through 4. The simulator allows you to run your programs without access to RISC-V hardware and allows the user to look up the register values, execute programs step-by-step and check the memory state.

The DTEK-V tools will be used for Assignments 5 through 7. These tools allow you to run your code on the DTEK-V board with riscv32. Assignment 5 describes how to get started with the DTEKV Tools.

All files required to perform the laboratory exercise are attached to the Canvas assignment **Lab 1 Files + Upload code**: the archive *lab1-files.zip* with four assembly files (*analyze.S*, *hex2asc.S*, *timetemplate.S* and *labmain.S*), and the board-support archive *time4riscv.zip* used from Assignment 5 onwards.

1.4 Reading Guidelines

Review the following course material while you are preparing your solutions.

- Lectures 1, 2, and 3.
- The course web page is named *Reading Guidelines* that lists specific chapters in the textbooks. See information about module 1.
- The *RISC-V reference sheet*, which is available from the *Literature* page on the course web site.

General description of the files in the compressed archive *time4riscv.zip* These files contain important functions that are needed to run on the DTEK-V board, which include how to boot up the board, how to manage interrupts, etc.

NOTE: These files do not contain a main (entry) function and thus are insufficient to make a working program. Instead, a user must insert lab-specific files. Please complement these with lab-specific files to make a working program.

- **COPYING**: Copyright information. Add lines with your name/names when you edit or add one or more files.
- **Makefile**: Standard file telling the system how to compile RISC-V programs. Don't change this file.
- **dtekv-lib.c**: Contains functions for printing characters. If you want to change this file, you are probably doing something wrong.
- **dtekv-lib.h**: The header containing the function prototypes in dtekv-lib.c.
- **boot.S**: Contains start-up code and interrupt-handling code. After initialization, this code calls the *main* function. Don't change this file.
- **dtekv-script.lds**: Describes how to assemble the compiled object files into a binary executable ready to be executed on the DTEK-V board.
- **softfloat.a**: This is a library that emulates floating-point operations, and is based on the high-performance Rvfplib (`https://github.com/pulp-platform/RVfplib/`).

2 Examination

The examination takes place during your booked lab session. The teachers examine your solutions at the lab computers: you run your programs, explain your code, and answer questions about it (the "Questions for Assignment" lists throughout this document show the kind of questions to expect). During the session you also solve a surprise assignment *as a pair* (see Assignment 7) and demonstrate the result to the teacher. The questions, however, are answered *individually*: the teacher addresses each of you in turn, and each of you must be able to answer on your own.

Both lab partners must be able to explain all parts of the solution. All programs must assemble without errors and warnings before the examination. See the course Canvas page for booking rules and deadlines.

This lab is examined in three parts. You can only be examined on one part at a time.

- Part 1 – Assignments 1 and 2.

- Part 2 – Assignments 3 through 6.
- Part 3 – Assignment 7, the surprise assignment.

When you pass a part, the teacher enters your grade in Canvas at the session. **Before you leave, immediately double-check under *Grades* that the teacher has registered your result** – if it is missing, tell the teacher right away.

Remember: upload your solution files to the Canvas assignment **Lab 1 Files + Upload code** *before* your booked session (the assignment page lists exactly which files). When you log in on the KTH lab computers, your submitted code is downloaded automatically – that is the code you present. If you modify it during the session, you may re-upload it to Canvas.

3 RARS Setup

When you run the following Assignments in the RARS simulator, make sure that **pseudo-instructions are turned on** and that the option for **Assembling all files currently open is selected.** To do that, go into *Settings* and make sure that the following checkboxes are selected:

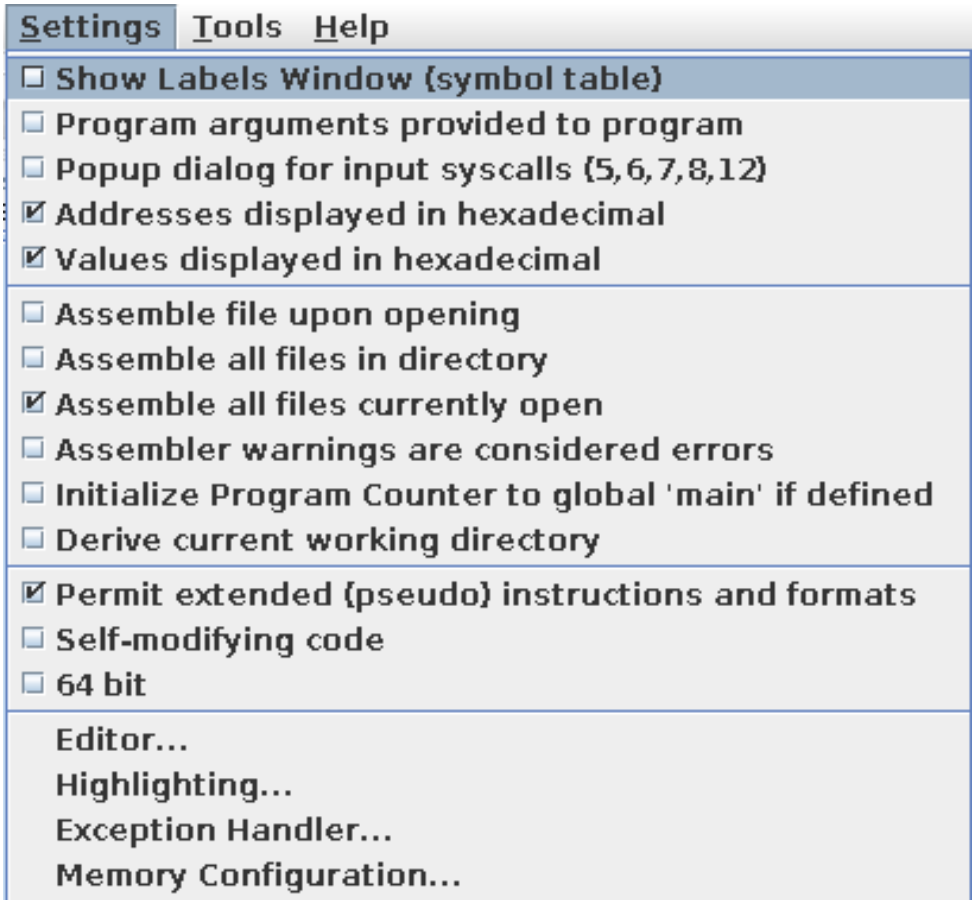


Figure 1: The recommended settings (on or off) for all options are shown in the screenshot.

Running an assignment in RARS

If you have not already done so, download *lab1-files.zip* from the Canvas assignment **Lab 1 Files + Upload code** and unpack it into a working directory. The archive contains the four

assembly files used in Assignments 1–4: *analyze.S*, *hex2asc.S*, *timetemplate.S*, and *labmain.S*.

labmain.S is the entry point that contains the `main:` label and a list of (commented-out) `jal` calls – one per assignment in this lab. For each assignment you work with exactly two files open in RARS: *labmain.S* and the file you are editing (*analyze.S*, *hex2asc.S*, or *timetemplate.S*).

Close any other tabs in RARS. The setting "Assemble all files currently open" assembles every open tab; an extra tab can introduce duplicate labels or shadow your symbols.

To run an assignment:

1. *File* → *Open* the two files.
2. In *labmain.S*, uncomment exactly one `jal` line – the one for the assignment you want to run (e.g. `jal analyze` for Assignment 1) – and leave the other `jal` lines commented out.
3. **Click on the *labmain.S* tab so it is the active editor.** This matters: RARS assembles starting from the active file. Running with *analyze.S* (or any other helper file) active produces an "*Instruction load access error*" because those files have no `main` label.
4. Press *F3* (or click the Assemble icon) to assemble.
5. Press *F5* (Go) to run at full speed, or *F7* (Step) to single-step.
6. *F12* resets the simulation; *F11* stops a running program.

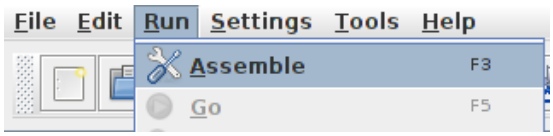


Figure 2: Assembling the code. Make sure that you're assembling with *labmain.S* selected in Editor.

4 Assignments

4.1 Assignment 1: Analyzing Assembly Code

Open *analyze.S* and *labmain.S* in RARS (close any other tabs). In *labmain.S*, uncomment `jal analyze`. Click on the *labmain.S* tab so it is the active editor, then assemble (F3). See *"Running an assignment in RARS"* above if you hit an "*Instruction load access error*" – the most common cause is forgetting to make *labmain.S* the active editor before assembling.

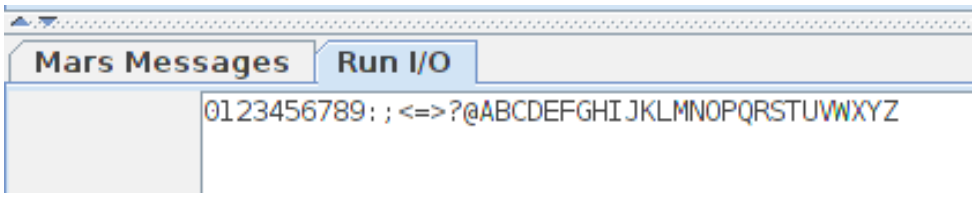


Figure 3: Expected output of the code in labmain.S.

File *analyze.S* contains a small function which prints the ASCII characters from '0' through 'Z' in the "Run I/O" console window in RARS. After 'Z', the program stops.

Single-step through the program with *F7*, then run it at full speed with *F5*. While single-stepping, please note how the contents of registers *s0* and *a0* change during execution.

Register values are shown in the **Registers** panel on the right side of the RARS window (each register is listed by both its ABI name *s0*, *a0*, ... and its numeric name *x8*, *x10*, ...).

After every *F7* step the register that was just modified is **highlighted** (the exact color depends on your RARS theme), so you can tell at a glance which instruction wrote which register.

Now, edit *analyze.S* so that only every third character is printed. The program must still stop after "Z".

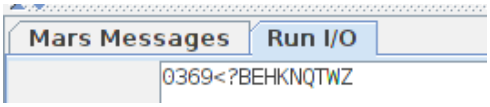


Figure 4: Expected output of the modified code in analyze.S (every third character printed).

Questions for Assignment 1 The following questions aim to help you check that you understand the code well. At the examination, the teacher may choose to ask questions which are not on this list.

- Which lines of code had to be changed? Why?

4.2 Assignment 2: Writing Your First Assembly Language Function

Open *hex2asc.S* (from the *lab1-files.zip* you already downloaded) in RARS, together with *labmain.S*. Close the *analyze.S* tab; in *labmain.S* comment out `jal analyze` and uncomment `jal hex2asc`.

You will now write a small subroutine, that converts numbers 0 through 15 into a printable ASCII-coded character: '0' through '9' or 'A' through 'F' depending on the number.

Only the 4 least-significant bits of *a0* are used as the input number; any higher bits are discarded. So input value 17 (binary `10001`) is treated as 1 and returns ASCII '1', input value 32 is treated as 0 and returns ASCII '0', and so on.

In file *hex2asc.S*, add an Assembly language subroutine with the following specification:

- *Name*: The subroutine must be called *hexasc*.
- *Parameter:* One, in register *a0*. The 4 least significant bits specify a number, from 0 through 15. All other bits in register *a0* can have any value and must be ignored.
- *Return value:* The 7 least significant bits in register *a0* must be an ASCII code as described below. All other bits must be zero when your function returns.
- *Required action:* The function must convert input values 0 through 9 into the ASCII codes for digits '0' through '9', respectively. Input values 10 through 15 must be converted to the ASCII codes for letters 'A' through 'F', respectively. An ASCII code table is available at the end of this document (see ASCII Table on p. 14).

Important note for all subroutines: When a subroutine returns, registers s1–s11, sp, s0 (or fp), ra, gp and tp **must** have the same contents as when the subroutine was called. If a subroutine uses any of these registers, the original contents must be saved, and restored before the subroutine returns. All other register contents may be modified by any subroutine.

The other direction matters too: you only need to save and restore a register if your function actually modifies it. Pushing/popping registers you never touch is wasted work and noise in the listing – keep the prologue and epilogue minimal.

If you are unsure how to save and restore registers, review the part of the assembly lecture that covers *calling functions and the stack* (reading guidelines: Lectures 1–3). The `PUSH` / `POP` macros provided in *timetemplate.S* (see Assignment 3) are useful for this.

Run the program and explain the output. Reminders before you assemble: make sure *labmain.S* has `jal hex2asc` uncommented (and `jal analyze` commented out), and that *labmain.S* is the active editor tab when you press *F3*. *(Note on naming:* `hex2asc` is a small wrapper already provided at the top of *hex2asc.S* – it loads a test value into `a0`, calls your *hexasc*, and prints the result. `labmain.S` calls the wrapper; the wrapper calls your function.*)*

Change the constant on the line marked "test number" (line 9 of *hex2asc.S*, currently `li a0, 4`) and re-run. Try all values from 0 through 15 and check that the digits '0' through '9' and the letters 'A' through 'F' appear correctly in the "Run I/O" console in RARS.

Questions for Assignment 2 The following questions aim to help you check that you understand the code well. At the examination, the teacher may choose to ask questions which are not on this list.

- **Hint:** a clean *hexasc* fits entirely in the temporary registers (`t0`–`t6`) and `a0`; it never touches an s-register, so it needs no `PUSH`/`POP` – the function has no prologue or epilogue, just the body.
- Your subroutine *hexasc* is called with an integer value as an argument in register `a0` and returns a value in register `a0`. If the argument is 17, what is the return value? Why?
- If your solution contains a conditional-branch instruction: which input values cause the instruction to branch to another location? This is called a *taken* branch.

4.3 Assignment 3: Printing the Time

In this assignment you will complete the clock template *timetemplate.S* so that it prints the current time once per iteration of its main loop. You will work through it in stages:

(a) Open *timetemplate.S* (from the *lab1-files.zip* you already downloaded). It contains most of the parts of a clock function.

The variable *mytime* is declared on line 38 of *timetemplate.S* (`mytime: .word 0x5957`) and initialised to 59:57, read as 59 minutes and 57 seconds. The subroutine *tick* (line 75) increments the contents of the *mytime* variable once for each iteration of the main loop (label *timetemplate:* on line 52, looping back via *j timetemplate* on line 71).

The variable *mytime* stores time-info as four separate decimal digits, each encoded in 4 bits as its natural binary value: digit '0' is `0000`, '1' is `0001`, ..., '9' is `1001`. This encoding is called **NBCD** (*Natural Binary-Coded Decimal*). The four other 4-bit patterns (`1010`...`1111`, i.e. values 10 through 15) are technically representable in 4 bits but are *not* valid NBCD digits and never appear in *mytime*.

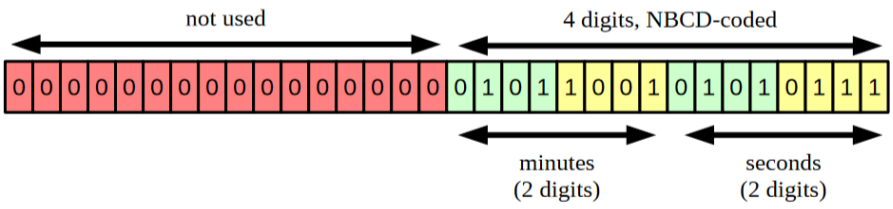


Figure 5: Time representation in a word: four 4-bit NBCD digits packed into the low 16 bits, in order tens-of-minutes / minutes / tens-of-seconds / seconds. Example: `0x1653` encodes 16:53.

The main loop in *timetemplate.S* already *calls* `delay` (line 58) and `time2string` (line 68), but neither subroutine is *defined* in the template – both labels are missing. You will complete the clock program by writing these two subroutines (parts (c) and (d) below).

(b) Copy your *hexasc* function (the one you wrote in Assignment 2 – the function whose label is `hexasc:`, in the file *hex2asc.S*). Paste the copied code at the end of the file *timetemplate.S*.

(c) Write a small function, like this, at the end of the file *timetemplate.S*:

```
delay:
  jr ra
```

This is a temporary version of the *delay* function. You will soon write a more useful version.

(d) You will now write a function that converts time-info into a string of printable characters, with a null-byte as an end-of-string-marker. At the end of file *timetemplate.S*, add an assembly-language subroutine with the following specification:

- *Name*: The subroutine must be called *time2string*.
- *Parameters (two):* Register *a0* contains the address of an area in memory, suitably large for the output from *time2string*. The 16 least significant bits of register *a1* contains time-info, organized as four NBCD-coded digits of 4 bits each. All other bits in register *a1* can have any value and must be ignored. *For example:* register *a0* can contain the address *0x100100017*, and register *a1* can contain the value *0x00001653*.
- *Return value:* None.
- *Required action:* The following sequence of six characters must be written to the area in memory pointed to by register *a0*:
  - Two ASCII-coded digits showing the number of minutes, according to the two more significant NBCD-coded digits of the input parameter. *Example:* '1', '6' (ASCII 0x31, 0x36).
  - A colon character (ASCII :, code 0x3A).
  - Two ASCII-coded digits showing the number of seconds, according to the two less significant NBCD-coded digits of the input parameter. *Example:* '5', '3' (ASCII 0x35, 0x33).
  - A null byte (ASCII NUL, code 0x00).

Pitfall: Even when calling your own subroutine *hexasc*, you must save and restore registers just as if you didn't know anything about the internals of *hexasc*.

NOTE: You **must** use the function *hexasc* to convert each NBCD-coded digit into the corresponding ASCII code. Use the *sb* instruction to store each byte at the destination. The macros *PUSH* and *POP* are useful for saving and restoring registers.

Important note for all subroutines (reminder): When a subroutine returns, registers s1–s11, sp, s0 (or fp), ra, gp and tp **must** have the same contents as when the subroutine was called. If a subroutine uses any of these registers, the original contents must be saved, and restored before the subroutine returns. All other register contents may be modified by any subroutine.

(e) Test your program using the RARS simulator (don't forget to run it from *labmain.S* with the jump to *timetemplate* uncommented). When testing your function, insert a breakpoint at the instruction *j timetemplate*: the breakpoint will cause RARS to pause your program at the breakpoint, just before jumping back for the next iteration.

Setting a breakpoint in RARS. After you assemble (F3), the main window switches to the *Execute* pane. The instructions appear as a table with columns *Bkpt* | *Address* | *Code* | *Basic* | *Source*, one row per instruction. The leftmost column, *Bkpt*, is a checkbox. Scroll to the row whose *Source* column shows `j timetemplate` and tick its checkbox. The

row is now marked as a breakpoint. Press *F5* (Go) and the simulator runs at full speed until it reaches that instruction, then halts so you can inspect registers and memory. Press *F5* again to continue to the next hit. To remove a breakpoint, untick the box; to clear all breakpoints, right-click the *Bkpt* column header and choose *Clear all breakpoints*.

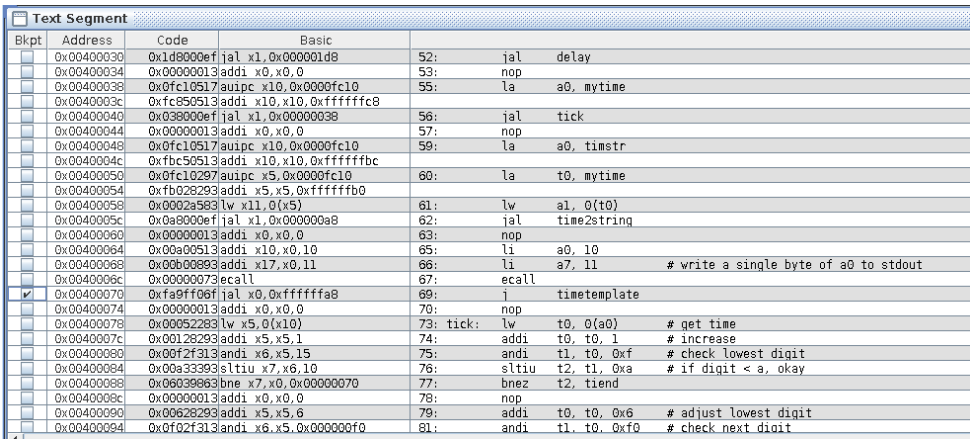


Figure 6: Setting up a breakpoint.

Questions for Assignment 3 The following questions aim to help you check that you understand the code well. At the examination, the teacher may choose to ask questions which are not on this list.

- **Hint:** go through your register usage carefully – the examiner will check that you save exactly the registers you actually modify, and no more.
- Which registers are saved and restored by your subroutine? Why?
- Which registers are used but not saved? Why are these not saved?
- Assume the time is 16:53. Which lines of your code handle the '5'?

4.4 Assignment 4: Programming a Simple Delay

You will now write a simple *delay* function. A routine like this lets a program pause between two actions – the clock you built in Assignment 3 uses it to make each tick visible to a human. Spinning the CPU in a busy loop is wasteful, though: the CPU does no useful work while it waits. In Lab 3 you will replace it with a much better version driven by a hardware timer. For now, three steps:

(a) Rewrite the following C function as an assembly-language subroutine with the same behavior.

```
void delay( int ms ) /* Wait a number of milliseconds, specified by
    the
                        parameter value. */
{
  int i;
  while( ms > 0 )
  {
    ms = ms - 1;
    /* Executing the following for loop should take 1 ms */
    for( i = 0; i < 4711; i = i + 1 ) /* The constant 4711 must be
        easy
```

```
                                                    to change! */
    {
      /* Do nothing. */
    }
  }
}
```

Important note for all subroutines (reminder): When a subroutine returns, registers s1–s11, sp, s0 (or fp), ra, gp and tp **must** have the same contents as when the subroutine was called. If a subroutine uses any of these registers, the original contents must be saved, and restored before the subroutine returns. All other register contents may be modified by any subroutine.

Note: the constant 4711 is too large to fit into the *addi* instruction; use *li* or equivalent instead.

(b) Replace the three-line *delay* subroutine from the previous assignment with your own assembly-language subroutine *delay* in file *timetemplate.S*.

(c) Test your program in RARS to confirm *delay* returns cleanly for several values of `a0` (e.g. 0, 1, 10, 1000). In RARS the simulator runs as fast as your laptop, so the literal wall-clock duration of one millisecond is meaningless here – you are only verifying control flow (the inner loop counts down, the outer loop runs the right number of times, and `a0=0` returns immediately without spinning). *Adjusting the constant for real 1 ms timing happens on the DTEK-V board in Assignment 6* – the usual procedure there is: call `delay(10000)` (i.e. a 10-second delay), time it against a phone stopwatch, then scale the constant by the ratio between expected and observed time, and iterate until 10 seconds runs in $10\,\mathrm{s} \pm 10\,\%$.

Questions for Assignment 4 The following questions aim to help you check that you understand the code well. At the examination, the teacher may choose to ask questions that are not on this list.

- **Hint:** walk through your assembly side-by-side with the C pseudo-code above and check that every operation in C has a corresponding sequence of instructions in your subroutine.
- If the argument value in register a0 is zero, which instructions in your subroutine are executed? How many times each? Why?
- Repeat the previous question for a negative number: -1.

4.5 Assignment 5: Move to the Development Board

For this assignment, you need the RISC-V cross-compiler (*riscv32-unknown-elf-gcc*) installed – it is what the `make` command below uses to build code for the board. The DTEK-V host tools (*dtekv-run*, *dtekv-upload*, *dtekv-download*) are *not* needed until Assignment 6, where *dtekv-run* uploads and runs your program on the board. Please see the course webpage "Software Resources" for installation instructions for both.

No DTEK-V board is needed for this assignment: everything here is cross-compilation on your own computer. You will run the result on the board in Assignment 6.

a) Unpack the *time4riscv.zip* you already downloaded into a working directory of your choice.

b) Copy your completed *timetemplate.S* (it contains your *hexasc*, *delay*, and *time2string* from Assignments 2–4) and *labmain.S* into the directory where *time4riscv.zip* was unpacked. Copy **only** these two files: do *not* copy *analyze.S* or *hex2asc.S*. The build compiles every assembly file it finds, and *hex2asc.S* contains a second copy of your *hexasc*, which would cause a duplicate-symbol error at link time.

In *labmain.S*, make sure that exactly one call is uncommented: `jal timetemplate`. If `jal analyze` or `jal hex2asc` is still uncommented, the build fails with an *undefined reference* – those files are not part of the board build.

A note on `.global`: in RARS, the setting "Assemble all files currently open" makes every label visible across files, so no declarations are needed. The GNU assembler and linker used here are stricter: a symbol referenced from another file must be declared with the *.globl* directive. The provided files already declare everything this build needs (`.globl timetemplate, tick, time2string, delay` in *timetemplate.S*, and `.globl main, handle_interrupt` in *labmain.S*) – but if you renamed a function or added a file of your own, you must add the declaration yourself.

c) In the Terminal window, change directory to the folder with the unzipped contents of file *time4riscv.zip*. Then type

```
make
```

You will see a few lines of messages. Look for warnings and errors, and correct them all. *Exception:* the linker warning "`main.elf has a LOAD segment with RWX permissions`" is expected on recent toolchains and is harmless on the DTEK-V board – you do not need to (and cannot) fix it.

Note: The assembly-language dialects are slightly different for RARS and GCC. The `PUSH` / `POP` macros in *timetemplate.S* ship in their RARS form (because Assignments 1–4 are done in RARS). For the DTEK-V build, open *timetemplate.S*, **comment out** the RARS macro block and **uncomment** the DTEK block directly below it. There are no other dialect differences in the lab code.

When the build succeeds, the directory contains *main.bin* (the binary for the board) and *main.elf.txt* (a disassembly listing you can inspect). That completes Assignment 5 – uploading and running *main.bin* on the board is Assignment 6.

Questions for Assignment 5 The following questions aim to help you check that you understand the code well. At the examination, the teacher may choose to ask questions that are not on this list.

- What is the effect of the assembler directive *.global*? Why is the directive particularly important in this assignment? The teachers will help you with this if necessary.

4.6 Assignment 6: At the Lab Session

For this assignment, you must have access to the DTEK-V toolchain and its binary *dtekv-run*. Please see the course webpage "Software Resources" for more information. In the Terminal window, at the prompt, change directory to the folder with your code.

a) Connect the DTEK-V board to your computer using the provided USB cable. Wait a few seconds and ensure that the board is active and lit up. After that, compile your program by typing:

```
make
```

followed by (assuming there were no errors in above compilation)

```
dtekv-run main.bin
```

to run your project on the board.
If this does not work immediately, you need to figure out why. Some possible candidates are:

- The jtagd daemon stopped working or timed-out. Type: `killall jtagd && jtagd --user-start` to restart it.
- Your compilation failed with errors. Please look through your code and fix the errors and try again.
- You have an existing application running on the DTEK-V board. Press the reset button (see Figure 7) and try again.

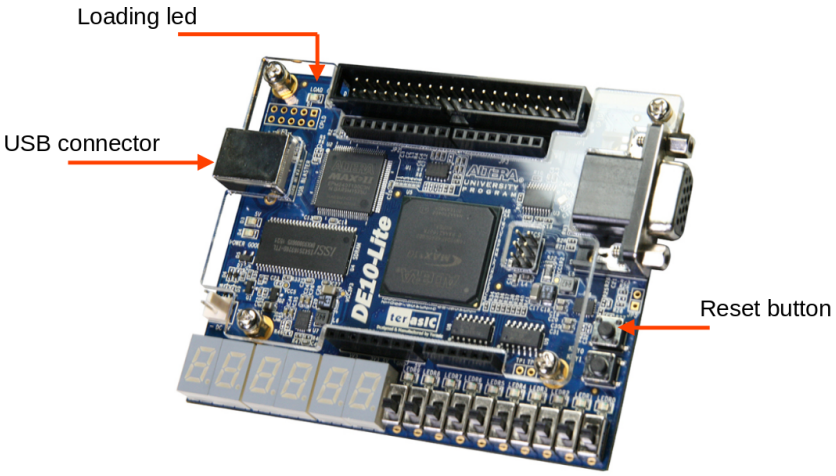


Figure 7: DTEK-V board: USB connector, Loading LED, and Reset button.

Ask the laboratory teachers for help if necessary and see the course Canvas page for further troubleshooting options.

During the time when the binary is being downloaded, information is being shown on your terminal and the LED called "Load" is blinking. When the process is done, your code should be running on your DTEK-V board.

NOTE: The command *make* will only compile the code, and the command *dtekv-run main.bin* will only download into the board whatever is already compiled. Therefore, you need to use both commands in order to compile and run an update of your code.

NOTE: Remember to Reset the device when you want to program it by pressing the reset button (KEY0).

b) Update the constant in the for loop of the delay subroutine, so that the time is updated correctly. Going from 00:00 to 01:59 should take two minutes. An error of less than +/- 10% is acceptable – that's 6 seconds per minute. Test and run your project on the board to adjust and demonstrate this.

Questions for Assignment 6 The following questions aim to help you check that you understand the assignment well. At the examination, the teacher may choose to ask questions that are not on this list.

- When you move your code from the simulator to the lab-board, you must change the value of the constant in the *delay* subroutine to get correct timing. Why? (**Hint**: the DTEK-V board runs at 30 MHz)

4.7 Assignment 7: Surprise Assignment

During the lab session, each pair receives a surprise assignment on the parts of Lab 1 you have prepared. The variants are randomly assigned. You solve the assignment *together as a pair* during the session and demonstrate the working result to the teacher. Questions about it are answered *individually*. The surprise assignment must run on the DTEK-V board.

5 ASCII Table – Hexadecimal Codes and Corresponding ASCII Characters

In this table, "0x" before a number indicates that the number is hexadecimal (in base 16).

0x00 NUL	0x20 SP	0x40 @	0x60 ‘
0x01 SOH	0x21 !	0x41 A	0x61 a
0x02 STX	0x22 "	0x42 B	0x62 b
0x03 ETX	0x23 #	0x43 C	0x63 c
0x04 EOT	0x24 $	0x44 D	0x64 d
0x05 ENQ	0x25 %	0x45 E	0x65 e
0x06 ACK	0x26 &	0x46 F	0x66 f
0x07 BEL	0x27 ’	0x47 G	0x67 g
0x08 BS	0x28 (	0x48 H	0x68 h
0x09 HT	0x29)	0x49 I	0x69 i
0x0A LF	0x2a *	0x4a J	0x6a j
0x0B VT	0x2b +	0x4b K	0x6b k
0x0C FF	0x2c ,	0x4c L	0x6c l
0x0D CR	0x2d -	0x4d M	0x6d m
0x0E SO	0x2e .	0x4e N	0x6e n
0x0F SI	0x2F /	0x4f O	0x6f o
0x10 DLE	0x30 0	0x50 P	0x70 p
0x11 DC1	0x31 1	0x51 Q	0x71 q
0x12 DC2	0x32 2	0x52 R	0x72 r
0x13 DC3	0x33 3	0x53 S	0x73 s
0x14 DC4	0x34 4	0x54 T	0x74 t
0x15 NAK	0x35 5	0x55 U	0x75 u
0x16 SYN	0x36 6	0x56 V	0x76 v
0x17 ETB	0x37 7	0x57 W	0x77 w
0x18 CAN	0x38 8	0x58 X	0x78 x
0x19 EM	0x39 9	0x59 Y	0x79 y
0x1a SUB	0x3a :	0x5a Z	0x7a z
0x1b ESC	0x3b ;	0x5b [	0x7b {
0x1c FS	0x3c <	0x5c \	0x7c |
0x1d GS	0x3d =	0x5d]	0x7d }
0x1e RS	0x3e >	0x5e ^	0x7e ~
0x1f US	0x3f ?	0x5f _	0x7f DEL

NUL null character, used as filler.

BEL bell, makes your computer beep when printed.

BS backspace, moves the cursor left one step.

LF line feed, moves the cursor down one line (often without moving to the leftmost column).

FF form feed, starts a new page on a hard-copy printout (and sometimes also on screen).

CR carriage return, moves the cursor to the beginning of the current line.

ESC escape, starts a sequence of control characters.

DEL delete, sometimes used to delete one character.

SP space between words.

SOH, STX, ETX, EOT, ENQ, ACK, HT, VT, SO, SI, DLE, DC1, DC2, DC3, DC4, NAK, SYN, ETB, CAN, EM, SUB, FS, GS, RS, US are control characters; we don't use them in this lab.

6 Version History

1.0 – First published version. 2015-09-04.

1.1 – Refined and condensed questions, updated lab code. 2016-01-15.

1.2 – Added a short note that all programs need to be error and warning free, before examination.

1.3 – Ported to RISC-V/32 platform. 2024-07.

1.4 – 2026 revision; this variant is examined at the lab session (no auto-grader). 2026.